SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

---

Code of Conduct:

I **M O N I K A . R (1 9 2 3 2 4 2 8 1** *) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Monika.R

QUESTION-1:

Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

Bloom's Level: BL3 – Apply

ANSWER:

```
mysql> CREATE DATABASE CollegeDBS;
Query OK, 1 row affected (0.01 sec)

mysql> USE CollegeDBS;
Database changed
mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY, Name VARCHAR(30), Department VARCHAR(20));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Course(CourseID VARCHAR(10) PRIMARY KEY, CourseName VARCHAR(30), Faculty VARCHAR(30));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Enroll(StudentID INT, CourseID VARCHAR(10), Marks INT, FOREIGN KEY(StudentID) REFERENCES Student(StudentID), FOREIGN KEY(CourseID) REFERENCES Course(C
ourseID));
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student VALUES(101,'Rahul','CSE'),(102,'Priya','ECE'),(103,'Arun','CSE'),(104,'Meena','IT');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------+------------+
| StudentID | Name  | Department |
+-----------+-------+------------+
|       101 | Rahul | CSE        |
|       102 | Priya | ECE        |
|       103 | Arun  | CSE        |
|       104 | Meena | IT         |
+-----------+-------+------------+
4 rows in set (0.00 sec)

mysql> INSERT INTO Course VALUES('C101','DBMS','Dr. Kumar'),('C102','Java','Dr. Rani'),('C103','Python','Dr. John');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Course;
+----------+------------+------------+
| CourseID | CourseName | Faculty    |
+----------+------------+------------+
| C101     | DBMS       | Dr. Kumar  |
| C102     | Java       | Dr. Rani   |
| C103     | Python     | Dr. John   |
+----------+------------+------------+
3 rows in set (0.00 sec)
```

```
mysql> INSERT INTO Course VALUES('C101','DBMS','Dr. Kumar'),('C102','Java','Dr. Rani'),('C103','Python','Dr. John');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Course;
+----------+------------+-----------+
| CourseID | CourseName | Faculty   |
+----------+------------+-----------+
| C101     | DBMS       | Dr. Kumar |
| C102     | Java       | Dr. Rani  |
| C103     | Python     | Dr. John  |
+----------+------------+-----------+
3 rows in set (0.00 sec)

mysql> INSERT INTO Enroll VALUES(101,'C101',85),(101,'C102',90),(102,'C101',78),(103,'C103',95),(104,'C102',88);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Enroll;
+-----------+----------+-------+
| StudentID | CourseID | Marks |
+-----------+----------+-------+
|       101 | C101     |    85 |
|       101 | C102     |    90 |
|       102 | C101     |    78 |
|       103 | C103     |    95 |
|       104 | C102     |    88 |
+-----------+----------+-------+
5 rows in set (0.00 sec)

mysql> SELECT s.Name,c.CourseName,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID JOIN Course c ON e.CourseID=c.CourseID;
+-------+------------+-------+
| Name  | CourseName | Marks |
+-------+------------+-------+
| Rahul | DBMS       |    85 |
| Priya | DBMS       |    78 |
| Rahul | Java       |    90 |
| Meena | Java       |    88 |
| Arun  | Python     |    95 |
+-------+------------+-------+
5 rows in set (0.01 sec)

mysql> SELECT Name FROM Student WHERE StudentID IN (SELECT StudentID FROM Enroll WHERE Marks>(SELECT AVG(Marks) FROM Enroll));
+-------+
| Name  |
+-------+
| Rahul |
| Arun  |
| Meena |
+-------+
```

QUESTION: 2

Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Bloom's Level: BL4 – Analyse

ANSWER:

RELATIONAL ALGEBRA:

R1 ← Student ⋈ Student.StudentID=Enroll.StudentID Enroll

R2 ← R1 ⋈ Enroll.CourseID=Course.CourseID Course

R3 ← σ Department='CSE' ∧ Marks>80 ∧ CourseName='DBMS' (R2)

Answer ← π Name, CourseName, Marks (R3)

In one line:

πName,CourseName,Marks(σDepartment='CSE'∧ Marks>80 ∧ CourseName='DBMS'(Student ⋈ Enroll ⋈ Course))

QUESTION: 3

Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create

ANSWER:

```
mysql> CREATE DATABASE StudentDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE StudentDB;
Database changed
mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY, Name VARCHAR(30),
    -> Department VARCHAR(20), Age INT, Marks INT);
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO Student VALUES(101,'Rahul','CSE',20,85),(102,'Priya','ECE',21,90),(103,'Arun','CSE',19,78)(104,'Meena','IT',20,88);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '(
104,'Meena','IT',20,88)' at line 1
mysql> INSERT INTO Student VALUES(101,'Rahul','CSE',20,85),(102,'Priya','ECE',21,90),(103,'Arun','CSE',19,78),(104,'Meena','IT',20,88);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student WHERE Department='CSE';
+-----------+-------+------------+------+-------+
| StudentID | Name  | Department | Age  | Marks |
+-----------+-------+------------+------+-------+
|       101 | Rahul | CSE        |   20 |    85 |
|       103 | Arun  | CSE        |   19 |    78 |
+-----------+-------+------------+------+-------+
2 rows in set (0.00 sec)

mysql> UPDATE Student SET Marks=95 WHERE StudentID=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------+------------+------+-------+
| StudentID | Name  | Department | Age  | Marks |
+-----------+-------+------------+------+-------+
|       101 | Rahul | CSE        |   20 |    95 |
|       102 | Priya | ECE        |   21 |    90 |
|       103 | Arun  | CSE        |   19 |    78 |
|       104 | Meena | IT         |   20 |    88 |
+-----------+-------+------------+------+-------+
4 rows in set (0.00 sec)
```

QUESTION:4

Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

Bloom's Level: BL4 – Analyze

OUTPUT:

```
mysql> SELECT * FROM Student;
+-----------+-------+------------+------+-------+
| StudentID | Name  | Department | Age  | Marks |
+-----------+-------+------------+------+-------+
|       101 | Rahul | CSE        |   20 |    95 |
|       102 | Priya | ECE        |   21 |    90 |
|       103 | Arun  | CSE        |   19 |    78 |
|       104 | Meena | IT         |   20 |    88 |
+-----------+-------+------------+------+-------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Student WHERE Department='CSE';
+-----------+-------+------------+------+-------+
| StudentID | Name  | Department | Age  | Marks |
+-----------+-------+------------+------+-------+
|       101 | Rahul | CSE        |   20 |    95 |
|       103 | Arun  | CSE        |   19 |    78 |
+-----------+-------+------------+------+-------+
2 rows in set (0.00 sec)

mysql> UPDATE Student SET Marks=95 WHERE StudentID=101;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----------+-------+------------+------+-------+
| StudentID | Name  | Department | Age  | Marks |
+-----------+-------+------------+------+-------+
|       101 | Rahul | CSE        |   20 |    95 |
|       102 | Priya | ECE        |   21 |    90 |
|       103 | Arun  | CSE        |   19 |    78 |
|       104 | Meena | IT         |   20 |    88 |
+-----------+-------+------------+------+-------+
4 rows in set (0.00 sec)
```

QUESTION: 5

Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate

OUTPUT:

```
mysql> USE CollegeDBS;
Database changed
mysql> SELECT s.Name,c.CourseName,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID JOIN Course c ON e.CourseID=c.CourseID;
+-------+------------+-------+
| Name  | CourseName | Marks |
+-------+------------+-------+
| Rahul | DBMS       |    85 |
| Priya | DBMS       |    78 |
| Rahul | Java       |    90 |
| Meena | Java       |    88 |
| Arun  | Python     |    95 |
+-------+------------+-------+
5 rows in set (0.00 sec)

mysql> SELECT Name FROM Student WHERE StudentID IN (SELECT StudentID FROM Enroll WHERE Marks>80);
+-------+
| Name  |
+-------+
| Rahul |
| Arun  |
| Meena |
+-------+
3 rows in set (0.01 sec)
```

QUESTION: 6

Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

Bloom's Level: BL6 – Create

ANSWER:

```
mysql> USE StudentDB;
Database changed
mysql> CREATE VIEW CSE_Students AS SELECT StudentID,Name,Marks FROM Student WHERE Department='CSE';
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM CSE_Students;
+-----------+-------+-------+
| StudentID | Name  | Marks |
+-----------+-------+-------+
|       101 | Rahul |    95 |
|       103 | Arun  |    78 |
+-----------+-------+-------+
2 rows in set (0.01 sec)
```

JUSTIFICATION:

- A View is a virtual table based on one or more tables.
- It provides restricted access by showing only the required columns and rows.
- It improves security by hiding unnecessary data and provides abstraction by simplifying complex queries.

QUESTION:7

Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Bloom's Level: BL4 – Analyze

ANSWER:

RELATIONAL ALGEBRA:

πName,CourseName,Marks(σDepartment='CSE'$\wedge$ Marks>80 $\wedge$ CourseName='DBMS'(Student $\bowtie$ Enroll $\bowtie$ Course))

SQL QUERY:

```
mysql> USE CollegeDBS;
Database changed
mysql> SELECT s.Name,c.CourseName,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID JOIN Course c ON e.CourseID
=c.CourseID WHERE s.Department='CSE' AND e.Marks>80 AND c.CourseName='DBMS';
+-------+------------+-------+
| Name  | CourseName | Marks |
+-------+------------+-------+
| Rahul | DBMS       |    85 |
+-------+------------+-------+
1 row in set (0.00 sec)
```

QUESTION: 8

Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

Bloom's Level: BL5 – Evaluate

ANSWER:

```
mysql> USE CollegeDBS;
Database changed
mysql> CREATE INDEX idx_studentid ON Enroll(StudentID);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT s.Name,c.CourseName,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID JOIN Course c ON e.CourseID
=c.CourseID WHERE e.Marks>80;
+-------+------------+-------+
| Name  | CourseName | Marks |
+-------+------------+-------+
| Rahul | DBMS       |    85 |
| Rahul | Java       |    90 |
| Arun  | Python     |    95 |
| Meena | Java       |    88 |
+-------+------------+-------+
4 rows in set (0.00 sec)
```

```
mysql> USE CollegeDBS;
Database changed
mysql> CREATE VIEW StudenttView AS SELECT s.StudentID,s.Name,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID;

Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM StudenttView;
+-----------+-------+-------+
| StudentID | Name  | Marks |
+-----------+-------+-------+
|       101 | Rahul |    85 |
|       101 | Rahul |    90 |
|       102 | Priya |    78 |
|       103 | Arun  |    95 |
|       104 | Meena |    88 |
+-----------+-------+-------+
5 rows in set (0.00 sec)
```

QUESTION: 9

Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

Bloom's Level: BL6 – Create

ANSWER:

```
mysql> SELECT s.Name,AVG(e.Marks) AS AvgMarks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID GROUP BY s.StudentID,s.
Name HAVING AVG(e.Marks)>(SELECT AVG(Marks) FROM Enroll);
+-------+----------+
| Name  | AvgMarks |
+-------+----------+
| Rahul |  87.5000 |
| Arun  |  95.0000 |
| Meena |  88.0000 |
+-------+----------+
3 rows in set (0.01 sec)
```

QUESTION: 10

Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

Bloom's Level: BL5 – Evaluate

ANSWER:

```
mysql> SELECT s.Name,c.CourseName,e.Marks FROM Student s JOIN Enroll e ON s.StudentID=e.StudentID JOIN Course c ON e.CourseID
=c.CourseID WHERE e.Marks>80;
+-------+------------+-------+
| Name  | CourseName | Marks |
+-------+------------+-------+
| Rahul | DBMS       |    85 |
| Rahul | Java       |    90 |
| Arun  | Python     |    95 |
| Meena | Java       |    88 |
+-------+------------+-------+
4 rows in set (0.00 sec)
```

JUSTIFICATION:

- JOIN is the best choice because the required data is stored in multiple related tables (Student, Enroll, and Course).
- It retrieves all the required information in a single query.
- It is generally more efficient than subqueries for this type of retrieval.
- Views are useful for simplifying repeated queries and restricting data access, but a JOIN is more appropriate here for fetching related data from multiple tables.

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation